Hoof Hoof Hoof……

timelimit: 1s

Since young age, Bob has liked train. He has many collections of train which is distinguished by model, year, and country. Since young age, Bob has a dream to become train operator, until his dream really comes true. Bob is really happy enjoying his job as train operator. His job is to manage the train by checking which carriage is full or broken. When he meets full carriage, he will divide the weight by half and append a new carriage either behind the full carriage, or in front of. Position of a new carriage is determined by the nearer weight compares to behind and in front of the full carriage. When he meets the broken carriage, he will trash it out, and count the lost of weights. By the time, the numbers of carriage grow up, also the job of Bob will be more tiring. Until, one moment, Bob is really exhausted with his job, and think that he needs help. He remembers that you are the programmer, and can help him to finish the job. Help Bob with his job!.

Problem Description

Bob is a train operator. A train consists of several carriages, each with a specific weight. Some carriages are Full (**F**), some are Broken (**X**), and some are normal **(A)**.

1. **Full Carriage (F):** Bob must split its weight W into two: The original carriage keeps one half, and a new carriage with the other half is added.
    - Place the new carriage next to the neighbor (front or back) whose weight is **closer** to the new carriage's weight.
    - If it's the first carriage, append it **behind**. If it's the last, append it in **front**.
    - If the weight difference is equal, the new carriage is appended **behind** the full carriage.
    - The denotation of Full Carriage, becomes **(N)**, after the operations.
2. **Broken Carriage (X):** Bob must remove this carriage entirely and record its weight as a "Lost Weight".

Input Format

- The first line consists of one integer N, where N is the numbers of carriage.
- The second line consists of N values representing the weight and the denoted letter.

Output Format

Show the total lost weight and the sequence of carriage after all operations.

Constraints

- $1 \leq N \leq 1000000$
- $5 \leq W \leq 10000$

Input

```
5
11 A
10 X
13 F
20 A
15 X
```

Output

```
Total Lost : 25.00
11.00 A 6.50 A 6.50 N 20.00 A
```

Explanation

- First, we have 2 broken carriages, and one full carriage.
- After iterations, we meet the broken carriages, and we trash it out, so the sequence becomes 11 A, 13 F, 20 A, 15 X, and total lost becomes 10.
- After the next iterations, we meet the full carriages, since 11 is nearer than 20 to 13, so we place a new carriage behind the 13, and performs division operation, so the sequence becomes 11 A, 6.50 A, 6.50 N, 20 A, 15 X, and total lost remains same
- Finally, we meet the broken carriages, and we trash it out, so the sequence becomes 11 A, 6.50 A, 6.50 N, 20 A and total lost becomes 25.

Input

```
8
11 X
10 F
15 F
15 A
6 X
7 A
27 A
19 F
```

Output

```
Total Lost : 17.00
5.00 A 5.00 N 7.50 N 7.50 A 15.00 A 7.00 A 27.00 A 9.50 N 9.50 A
```

The Electrician's Obsession

timelimit: 1s

Kael is a brilliant programmer who is deeply obsessed with Physics, specifically the **Magic Triangle** of electricity. In his world, a "Magic Triangle" is a set of three values (V, I, R) that satisfy the equilibrium equation:

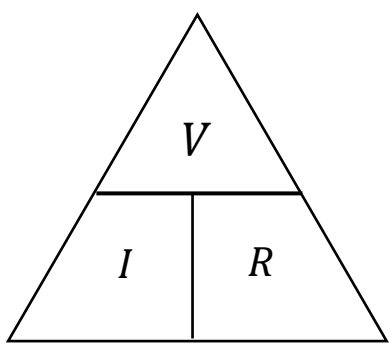
$$V = I \ x \ R$$

$(Where\ V\ is\ Voltage, I\ is\ current, and\ R\ is\ Resistor)$

On his birthday, Kael was gifted a **Decoration Tree**. He wishes to determine the total number of **Magic Triangles** hidden within the tree's structure.

Problem Description

A **Decoration Tree** in the form of a **Binary Search Tree (BST)**. Each node in this tree contains integer values, and the letter (V, I, R). A **Magic Triangle** is found if, for any node with letter V is the parent, I is the left child, and R is the right child.



Input Format

- The first line consists of one integer N, where N is the numbers of Node
- The second line consists of N values representing M integer, and the letter

Output Format

Show the number of magic triangles

Constraint

- $1 \leq N \leq 100000$
- $1 \leq M \leq 10^8, unique$

Input

```
9
20 V
10 V
2 I
5 R
50 V
55 V
11 I
7 R
1 I
```

Output

```
0
```

Explanation

- **20 V**: Becomes the **Main Root**.
- **10 V**: Inserted to the left of 20 (10 < 20).
- **2 I**: Inserted to the left of 10 (2 < 10).
- **5 R**: Inserted to the left of 20, left of 10 and right of 2 (5 < 20 and 5 < 10 and 5 > 2).
- **50 V**: Inserted to the right of 20 (50 > 20).
- **55 V**: Inserted to the right of 20, then to the right of 50 (55 > 20 and 55 > 50).
- **11 I**: Inserted to the left of 20, then to the right of 10 (11 < 20 and 11 > 10).
- **7 R**: Inserted to the left of 20, left of 10, right of 2, and right of 5 (7 < 20 and 7 < 10 and 7 > 2 and 7 > 5).
- **1 I**: Inserted to the left of 20, left of 10, left of 2 (1 < 20 and 1 < 10 and 1 < 2).

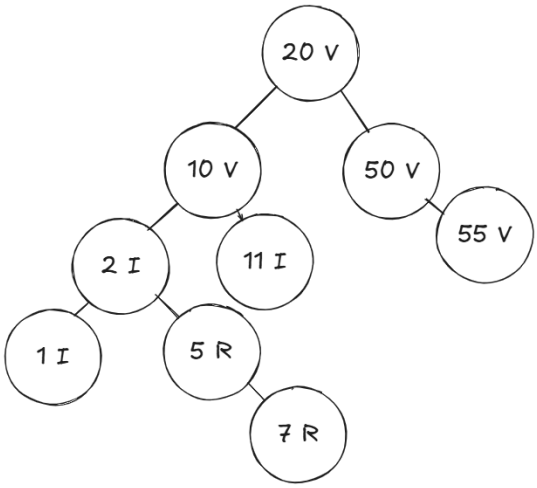


Figure 1 BST

Voltage Sensor Reading

timelimit: 1s

Kael has upgraded his **Decoration Tree** into a **Laboratory Monitoring System**. Each node in the tree now represents a **Voltage Sensor Reading** from his experiments. Because the data comes in real-time, Kael constantly **inserts** new readings into his **Binary Search Tree (BST)**.

To ensure his experiment is stable, Kael needs to monitor the extremes. Specifically, after new readings are inserted, he wants to find:

1. The **K-th Smallest** voltage reading.
2. The **K-th Highest** voltage reading.

Can you help Kael find these two values quickly?

Input Format

- The first line contains two integers, N (initial number of readings) and K (the rank to find).
- The second line contains N integers representing the initial voltage readings to be inserted into the BST.
- The third line contains an integer M, the number of A (**new** readings) to be inserted.
- The fourth line contains M integers, A (the new readings) that must be added to the existing BST.

Output Format

- Print two integers separated by a space: the **K-th Smallest** and the **K-th Highest** value after each insertions.
- If K is greater than the total number of nodes, print -1.

Constraint

- $1 \leq N \leq 1000$
- $1 \leq K \leq 10^8$
- $1 \leq M \leq 1000$
- $1 \leq A \leq 10^8, unique$

Input

```
5 2
10 20 30 40 50
2
15 45
```

Output

```
15 40
15 45
```

Explanation

- Initial tree: {10, 20, 30, 40, 50}.
- New readings inserted: 15.
- The final sorted readings are: {10, 15, 20, 30, 40, 50}.
- The **2nd (K=2) Smallest** is **15**.
- The **2nd (K=2) Highest** is **40**.
- New readings inserted: 45
- The final sorted readings are: {10, 15, 20, 30, 40, 45, 50}.
- The **2nd (K=2) Smallest** is **15**.
- The **2nd (K=2) Highest** is **45**.

Who's My Uncle!

timelimit: 1s

After successfully organizing his **Decoration Tree**, Kael starts to wonder about the "family" relationships between the nodes. He realizes that in a **Binary Search Tree (BST)**, every node (except the root) has a parent, and some even have an **Uncle**.

In Kael's tree, a node U is considered the **Uncle** of node X if:

1. P is the **direct parent** of X.
2. G is the **direct parent** of P (the grandparent of X).
3. U is the **other child** of G (the sibling of P).

Kael will give you a series of voltage readings to build a BST. Then, he will ask you to identify the **Uncle** of a specific node value. Can you help Kael find the missing relative?

Input Format

- The first line consists of an integer N, the number of nodes to be inserted into the BST.
- The second line consists of N integers representing the values to be inserted.
- The third line consists of an integer T, the numbers of node, that you have to find the uncle.
- The fourth line contains T integers, the values of the node whose **Uncle** you need to find.

Output Format

- Print the value of the **Uncle** of **X** node.
- If **X** node has no uncle, or if **X** node is the root/has no grandparent, print **-1**.
- If **X** node does not exist in the tree, print **Not Found**.

Constraint

- $1 \leq N \leq 10000$
- $1 \leq T \leq N$

Input

```
5
20 10 30 5 15
1
5
```

Output

```
30
```

Explanation

1. Insert **20** (Root).
2. Insert **10** (Left child of 20).
3. Insert **30** (Right child of 20).
4. Insert **5** (Left child of 10).
5. Target is **5**.
    - Parent of 5 is **10**
    - Grandparent of 5 is **20**
    - 20 has other child is **30**
    - So, **30 is the uncle of 5**

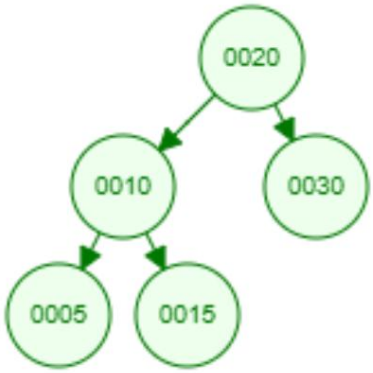


Figure 2 BST Visualization

Input

```
5
20 10 30 5 15
1
20
```

Output

```
-1
```

BONUS

https://www.hackerrank.com/challenges/balanced-brackets/problem

https://leetcode.com/problems/number-of-visible-people-in-a-queue/description/

https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=8&page=show_problem&problem=3139

https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=8&page=show_problem&problem=1113